Hardware & Software for Big Data

Francesco Fossari - Matricola D03000221

Challenge Description

This project originates from an assignment focused on Sentiment Analysis, requiring the development of a multiclass classification system applied to a large collection of tweets.

The dataset selected for the implementation is publicly available on Kaggle (https://www.kaggle.com/datasets/tariqsays/sentiment-dataset-with-1-million-tweets) and contains approximately one million tweets, each labeled with one of four sentiment categories:

- Positive
- Negative
- Litigious
- Uncertainty

Each record in the dataset includes three main fields:

- the text of the tweet
- the language of the tweet
- the associated sentiment label

The goal of the challenge is to analyze the tweets and assign the correct sentiment label to each instance.

The dataset provides pre-labeled examples, which are used to train and evaluate a Machine Learning model.

Subsequently, a portion of the dataset is treated as an unlabeled data stream, simulating a real-time scenario in which the model must generate predictions on previously unseen tweets.

The assignment explicitly requires the use of:

- Apache Kafka as the streaming ingestion and message-brokering system
- Apache Spark (Python API) for distributed processing, text preprocessing, model training, and real-time inference

The complexity of the challenge lies not only in building an effective classification model, but in integrating multiple heterogeneous components into a single, coherent end-to-end pipeline.

The project is designed to cover the entire data lifecycle, including:

- ingestion and streaming management through Kafka
- large-scale text preprocessing and feature engineering
- distributed model training
- evaluation using standard performance metrics
- deployment of the trained model in a streaming environment
- real-time prediction on unseen data

The final objective is to demonstrate the ability to design a complete, scalable, and robust system capable of operating in both batch and streaming modes, effectively replicating a realistic scenario of automated sentiment analysis on social media data.

Description of the Proposed Methodology & Architecture

2.1 Dataset Preparation

The dataset used in this project consists of approximately one million tweets, each labeled with one of four sentiment categories: Positive, Negative, Litigious, and Uncertainty.

Every record includes three key fields:

- the language of the tweet
- the tweet text
- the associated sentiment label

Before any modeling phase could begin, it was necessary to perform a series of data preparation steps to ensure the consistency, quality, and reproducibility of the entire pipeline.

The first preprocessing step involved language filtering. Since the model was designed to operate exclusively on English text, all tweets written in other languages were removed from the dataset.

This choice helps avoid semantic ambiguity and ensures a more homogeneous vocabulary, ultimately improving the effectiveness of text preprocessing techniques and downstream feature transformations.

After the initial preprocessing steps, the dataset was shuffled to eliminate any patterns related to the original ordering of the records.

This operation ensures a more uniform distribution of classes across the subsequent phases and prevents the model from learning spurious correlations tied to the dataset's original structure.

Following the shuffle, the dataset was divided into three distinct subsets:

- 50% for training, used to fit the classification model
- 30% for testing, used to evaluate model performance on previously unseen data
- 20% for streaming, treated as an unlabeled real-time data stream to be classified on the fly

This split directly satisfies the requirements of the assignment, which mandates the simulation of a real streaming environment using a portion of the dataset without labels.

Such a design makes it possible to assess not only the quality of the trained model, but also its ability to generalize to new data and integrate seamlessly into a streaming architecture built with Kafka and Spark.

The dataset preparation phase therefore represents a critical step in the project: it defines the scope of the problem, ensures the quality of the input data, and establishes the foundation for a coherent workflow that spans both batch processing and real-time processing.

2.2 Preprocessing Pipeline

Text analysis represents a crucial phase in the development of a Sentiment Analysis model, especially when working with short, informal, and noisy data such as tweets.

To ensure a coherent and scalable transformation of the input data, a preprocessing pipeline was designed entirely using Apache Spark MLlib, guaranteeing full consistency across the training, testing, and streaming stages.

The pipeline consists of a sequence of transformations that progressively convert raw text into a numerical representation suitable for training a Support Vector Machine (SVM) classifier.

Each component of the pipeline was selected to address a specific characteristic of Twitter language, such as informal grammar, abbreviations, hashtags, mentions, and high lexical variability, while maintaining efficiency at large scale.

Tokenization

The first stage of the preprocessing pipeline involves splitting the text into individual tokens using RegexTokenizer.

This component isolates meaningful words and symbols while automatically removing punctuation and non-alphanumeric characters.

Tokenization is a fundamental step, as it transforms raw text into a structured sequence of analyzable units, enabling all subsequent linguistic and statistical transformations.

Stopwords Remover

Next, the pipeline applies StopWordsRemover, which filters out the most common English words (such as “the”, “and”, “is”) that carry little to no discriminative value for sentiment classification.

Removing these high-frequency, low-information terms reduces noise in the dataset and improves the overall quality of the features generated in the subsequent stages of the pipeline.

Bigrams Generation

To capture local relationships between words, the pipeline employs NGram with n = 2, generating bigrams.

This choice is particularly effective in the context of tweets, where many sentiment-bearing expressions arise from two-word combinations such as “not good”, “very bad”, or “high risk”.

By modeling these short linguistic patterns, the system is able to detect nuances that would be lost when considering individual words in isolation.

HashingTF

The next stage transforms the tokens into numerical vectors using HashingTF, configured with a feature space of 100,000 dimensions.

This technique applies the hashing trick to map words and n-grams into a fixed-size vector, enabling the representation of highly diverse texts within a common vector space.

By avoiding the need to build and store a full vocabulary, HashingTF ensures both efficiency and scalability, making it particularly suitable for large-scale datasets such as the one used in this project.

IDF (Inverse Document Frequency)

The vector produced by HashingTF is then re-weighted using IDF (Inverse Document Frequency), which decreases the influence of overly frequent terms while amplifying the contribution of more discriminative ones.

This step enhances the model's ability to distinguish between the different sentiment classes by emphasizing words and expressions that carry meaningful informational value.

Normalizer

To prevent text length from influencing the model, the pipeline applies a Normalizer, which scales all feature vectors to unit norm.

This step ensures that longer tweets do not disproportionately affect the learning process and is particularly beneficial for linear models such as SVM, which are sensitive to differences in vector magnitude.

StringIndexer

Finally, the textual sentiment label is converted into a numerical index using StringIndexer, enabling the training of a multiclass classification model.

The resulting preprocessing pipeline provides several key advantages:

- consistency across training, testing, and streaming phases
- scalability to large-scale datasets
- robustness against the noise and variability typical of Twitter data
- compatibility with high-dimensional linear models such as SVM

Moreover, this modular structure allows the entire pipeline to be saved as a single Spark object, making it straightforward to reuse the trained model during the streaming phase without manually replicating the preprocessing steps.

2.3 Machine Learning Model

The choice of the machine learning model represents a central element in the design of the entire pipeline.

In the context of Sentiment Analysis on short texts such as tweets, it is essential to adopt a classifier capable of handling high-dimensional, sparse, and informal textual data.

For these reasons, the model selected for this project is the Linear Support Vector Machine (LinearSVC), integrated within a One-vs-Rest (OvR) strategy to support multiclass classification.

The LinearSVC classifier is particularly well-suited for this task due to its robustness in high-dimensional feature spaces, its efficiency on large datasets, and its strong performance on text classification problems where features derived from TF-IDF representations tend to be sparse and numerous.

LinearSVC

LinearSVC is a linear model particularly well-suited for text classification tasks.

Its main strengths include:

- effectiveness on high-dimensional data, typical of TF-IDF representations
- robustness to noise, which is frequent in informal text such as tweets
- fast training times, even on large-scale datasets
- strong generalization ability, especially when working with sparse feature vectors

The model relies on the OWL-QN optimizer, a variant of the L-BFGS algorithm specifically designed to handle L1 regularization.

This form of regularization encourages sparsity in the learned weights, helping reduce the impact of less informative features and improving the interpretability and efficiency of the classifier.

One-vs-Rest Strategy

Since LinearSVC is inherently a binary classifier, the One-vs-Rest (OvR) strategy was adopted to extend it to a four-class problem.

In this approach, a separate binary classifier is trained for each sentiment category.

Each model learns to distinguish one specific class from all the others, and the final prediction is assigned based on the classifier that produces the highest decision margin.

This strategy is particularly effective in scenarios where:

- the classes are not perfectly balanced
- the decision boundaries are approximately linear
- the dataset is large and requires computationally efficient models

The OvR framework therefore provides a practical and scalable solution for multiclass sentiment classification in high-dimensional text settings such as Twitter data.

Model Parameters

To ensure a good balance between accuracy and training time, the following hyperparameters were selected:

- maxIter = 120

Number of iterations for the OWL-QN optimizer, sufficient to guarantee stable convergence.

- regParam = 0.005

Regularization strength, chosen to reduce overfitting while maintaining strong generalization performance.

- tol = 1e-6

Convergence tolerance threshold.

These values were determined through preliminary experiments, with the goal of obtaining a model that is stable, efficient, and accurate, even when trained on a large-scale, high-dimensional dataset.

Pipeline Integration

The model is integrated as the final stage of the Spark ML pipeline, following all text-processing transformations.

This design choice provides two fundamental advantages:

- Consistency across training, testing, and streaming

The same preprocessing steps are automatically applied in every phase, ensuring that the model receives data in a uniform format.

- Model portability

The entire pipeline, both preprocessing and classifier, is saved as a single PipelineModel object, making it straightforward to reuse during the streaming phase without manually replicating the transformation logic.

2.4 System Architecture

The designed architecture integrates Apache Kafka and Apache Spark into a unified workflow that covers all stages of the data lifecycle: ingestion, preprocessing, training, evaluation, and real-time inference.

The objective is to build a system that is scalable, reproducible, and fully aligned with the principles of modern Big Data platforms.

By combining distributed stream processing with large-scale machine learning, the architecture ensures consistent behavior across both batch and streaming environments, enabling reliable sentiment classification on high-volume social media data.

Kafka Producer

Kafka is used as the data ingestion system within the architecture.

Two separate producers are responsible for sending tweets to the appropriate topics:

- batch producer → training and test data

- realtime producer → streaming data (20% of the dataset)

Examples of commands used during setup and execution include:

```
bin/zookeeper-server-start.sh config/zookeeper.properties

bin/kafka-server-start.sh config/server.properties
```

```
bin/kafka-topics.sh --create --topic tweets-stream --bootstrap-server localhost:9092
```

```
python3 SentimentalAnalysis/producer_realtime.py
```

Kafka Broker

The Kafka broker is responsible for managing several core components of the ingestion layer, including:

- message queues
- offset tracking
- partitioning
- fault tolerance

It represents the central node of the data ingestion process, ensuring reliable, ordered, and scalable delivery of messages to downstream consumers such as Spark Streaming.

Spark Batch - Training

The model is trained by reading the input data directly from Kafka and applying the full Spark ML pipeline.

The training job is executed using the following command:

```
spark-submit \

  --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.4.1 \

  SentimentalAnalysis/spark_train.py
```

Once training is completed, the final model is saved as a PipelineModel, allowing it to be seamlessly reused during the streaming phase without re-implementing the preprocessing steps.

Spark Batch - Test

The model is evaluated on 30% of the dataset using the following command:

```
spark-submit \

--packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.4.1 \

SentimentalAnalysis/spark_test.py
```

During the evaluation phase, the system computes the main performance metrics for multiclass classification, including accuracy, F1-score, precision, recall, and the confusion matrix.

These metrics provide a comprehensive assessment of the model's ability to correctly classify sentiment across all classes.

Spark Structured Streaming

The streaming component is responsible for consuming tweets in real time from the tweets-stream topic.

The streaming job is executed using the following command:

```
spark-submit \

--packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.4.1 \

SentimentalAnalysis/spark_streaming.py
```

Its main functionalities include:

- continuous ingestion of messages from Kafka
- application of the saved PipelineModel
- real-time sentiment prediction for each incoming tweet
- structured output printing through the foreachBatch function.

This component enables low-latency inference and ensures that the same preprocessing and classification logic used during training is consistently applied in the streaming environment.

CheckPointing

Spark uses a dedicated checkpoint directory:

```
SentimentalAnalysis/checkpoint/
```

This directory is essential for:

- storing Kafka offsets
- ensuring exactly-once processing semantics
- restoring the streaming query after a crash or unexpected interruption

Checkpointing allows the streaming application to resume from the last consistent state, guaranteeing reliability and fault tolerance throughout the real-time processing pipeline.

Architectural Paradigm

The system follows the principles of the Kappa Architecture, since:

- batch and streaming processing share the same ML pipeline
- Kafka acts as the single entry point for all incoming data
- Spark Structured Streaming handles continuous processing
- no duplication exists between a batch layer and a speed layer, unlike in the Lambda Architecture

The same processing logic is applied both to historical data (training/test) and to real-time data (streaming), resulting in a system that is simple, consistent, and easy to maintain.

Experimental Results

This chapter presents the results obtained in the three main phases of the project: training, testing, and real-time streaming.

Each phase was executed using Apache Spark and Apache Kafka, following the pipeline described in the previous chapters.

The terminal screenshots illustrate the actual execution of the commands, the model's output, and the behavior of the system under real operating conditions.

Training Results

The model was trained on 50% of the dataset, using the full preprocessing pipeline (tokenization, stopword removal, n-grams, HashingTF, IDF, normalization, and the One-vs-Rest classifier with LinearSVC).

During training, Spark reported stable convergence of the OWL-QN optimizer, with short iteration times and no memory-related issues, confirming the efficiency and scalability of the chosen configuration.

Test Results

The model was evaluated on 30% of the dataset reserved for testing.

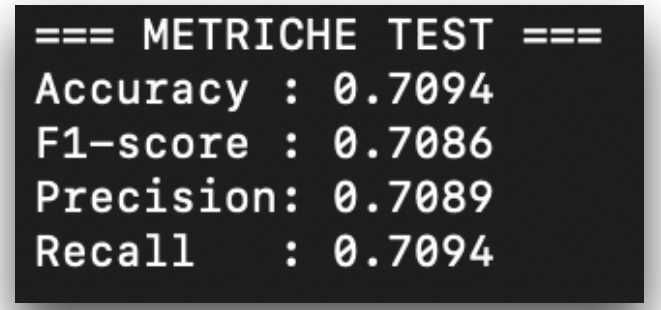
During this phase, the main classification metrics were computed:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

These metrics provide a comprehensive assessment of the model's ability to correctly distinguish among the four sentiment classes, highlighting both overall performance and class-specific behavior.

true_label	pred_label	count
litigious	litigious	31775
litigious	negative	3861
litigious	positive	2970
litigious	uncertainty	2740
negative	litigious	5455
negative	negative	37577
negative	positive	6846
negative	uncertainty	6050
positive	litigious	3622
positive	negative	5597
positive	positive	43169
positive	uncertainty	4839
uncertainty	litigious	3718
uncertainty	negative	5850
uncertainty	positive	6571
uncertainty	uncertainty	29360

Matrice di Confusione



Metriche di Test

Streaming Results

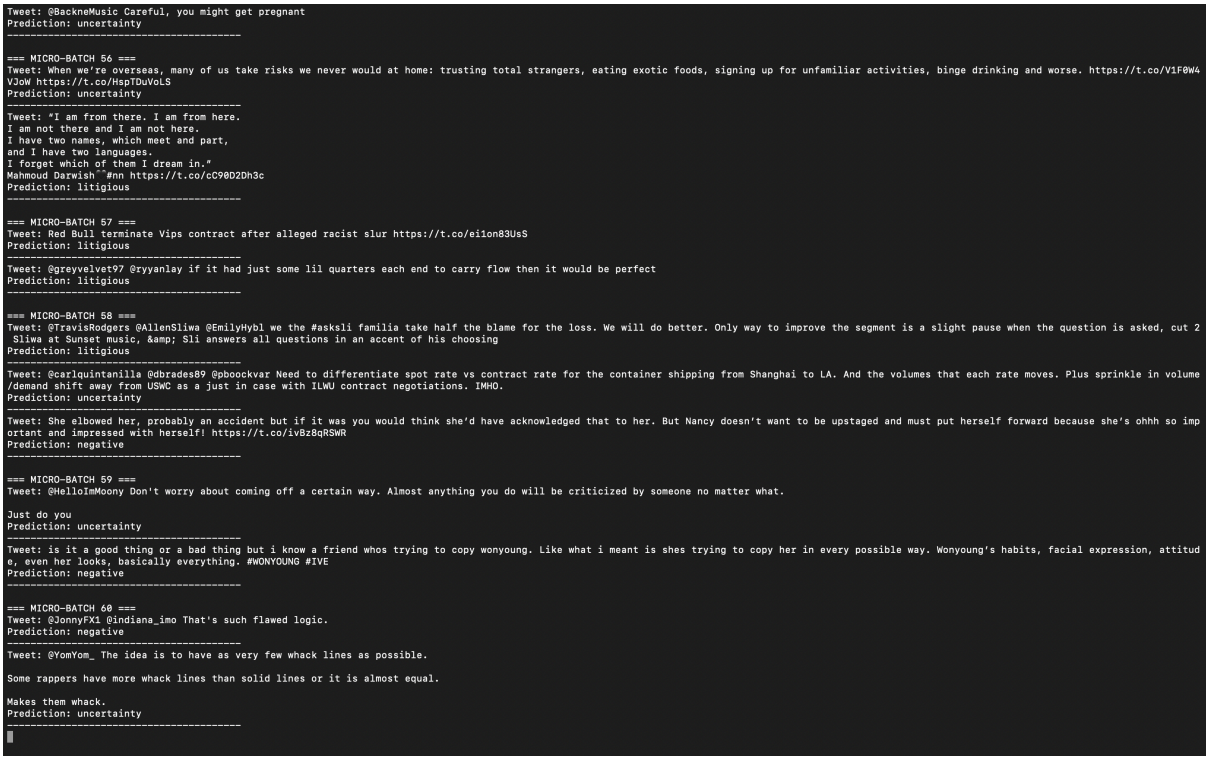
The streaming phase represents the most important part of the project, as it demonstrates the system's ability to operate in real time.

A 20% portion of the dataset was used as an unlabeled stream and sent to the Kafka tweets-stream topic through the realtime producer.

Spark Structured Streaming:

- continuously read the incoming tweets
- automatically applied the saved ML pipeline
- generated the corresponding sentiment predictions
- printed the results in micro-batches

This phase validates the end-to-end functionality of the architecture, showing that the system can ingest, process, and classify data with low latency under real operating conditions.



Final Considerations on Results

The model proved to be effective in both the batch and streaming phases.

The evaluation metrics confirm a solid generalization capability, while the integration with Kafka and Spark Structured Streaming enabled the development of a complete, robust, and production-ready system for real-world sentiment analysis on social media data.